Sprawozdanie z projektu
Tester sprawności psychomotorycznej w bibliotece PyQt6

Tomasz Nazar 197613, Filip Pietrzak 198275

30 marca 2026

1 Sformułowanie zadania i założenia szczegółowe

Celem projektu było stworzenie aplikacji okienkowej dla komputerów klasy PC, pełniącej funkcję testera sprawności psychomotorycznej (np. dla kandydatów na kierowców). Aplikacja została napisana w języku Python z wykorzystaniem biblioteki graficznej PyQt6.

1.1 Założenia funkcjonalne

Aplikacja realizuje trzy rodzaje testów:

1. **Prosty czas reakcji (bodziec optyczny):** Reakcja na zmianę koloru ekranu na zielony.
2. **Złożony czas reakcji (bodziec optyczny z wyborem):** Test typu Go/No-Go. Badany musi zareagować na bodziec docelowy (czerwony kwadrat) i zignorować dystraktor (zielone koło). Czas na reakcję jest limitowany (1.5 sekundy).
3. **Prosty czas reakcji (bodziec akustyczny):** Reakcja na sygnał dźwiękowy przy braku bodźców wizualnych.

Każdy z testów składa się z fazy szkoleniowej (3 próby) bez zapisu wyników oraz fazy właściwej (8 prób). Wyniki są prezentowane w formie syntetycznej (wartości średnie) oraz graficznej (wykres liniowy).

1.2 Sposób pomiaru czasu i określania interwałów

Do pomiaru czasu reakcji przyjęto użycie funkcji `time.perf_counter()`, która zapewnia najwyższą dostępną rozdzielczość zegara sprzętowego (rzędu ułamków mikrosekund) i jest odporna na ewentualne zmiany czasu systemowego w trakcie działania programu. Zastosowanie `time.time()` lub klasycznych obiektów `QTimer` do bezpośredniego mierzenia czasu reakcji uznano za niewystarczające.

Interwały czasowe pomiędzy bodźcami są generowane losowo za pomocą funkcji `random.uniform()` (np. w przedziale od 1.5 do 5.0 sekund). Losowość ta zapobiega wyuczeniu się rytmu przez badanego, wymuszając nieustanne skupienie uwagi.

2 Opis przyjętych rozwiązań programowych

2.1 Architektura interfejsu (QStackedWidget)

Aby uniknąć otwierania wielu nakładających się na siebie okien, zastosowano wzorzec stosu ekranów obsługiwany przez klasę `QStackedWidget`. Wszystkie widoki dziedziczą po `QWidget` i są rejestrowane w klasie głównej `MainWindow`.

```
self.stacked_widget = QStackedWidget()
self.setCentralWidget(self.stacked_widget)

# Rejestracja ekranow
self.menu_screen = MenuScreen(self)
self.simple_test = SimpleOpticalTest(self)

self.stacked_widget.addWidget(self.menu_screen)
self.stacked_widget.addWidget(self.simple_test)
```

Listing 1: Zarządzanie ekranami za pomocą QStackedWidget

2.2 Mechanizm precyzyjnego pomiaru czasu

Główny mechanizm pomiaru czasu opiera się na zapisaniu momentu wyświetlenia bodźca oraz przechwyceniu zdarzenia wciśnięcia klawisza spacji (`keyPressEvent`). Ewentualne wciśnięcia klawisza przed pojawieniem się bodźca (falstarty) są ignorowane lub komunikowane użytkownikowi dzięki fladze `self.is_waiting_for_reaction`.

```
def show_stimulus(self):
    self.setStyleSheet("background-color: #27ae60;")
    self.info_label.setText("WCISNIJ SPACJE!")
    self.start_time = time.perf_counter() # Start stopera
    self.is_waiting_for_reaction = True

def keyPressEvent(self, event):
    if event.key() == Qt.Key.Key_Space:
        if self.is_waiting_for_reaction:
            # Rejestracja czasu z najwyzsza precyzja
            reaction_time = time.perf_counter() - self.start_time
            self.is_waiting_for_reaction = False
            self.handle_reaction(reaction_time)
```

Listing 2: Logika precyzyjnego pomiaru czasu reakcji

2.3 Obsługa asynchroniczna testu Go/No-Go

W złożonym teście optycznym wykorzystano dodatkowy obiekt `QTimer` odliczający limit czasu (1500 ms). Jeżeli użytkownik nie zareaguje na dystraktor (zgodnie z instrukcją), zdarzenie `timeout` zalicza próbę jako poprawną, trenując powstrzymanie reakcji.

2.4 Generowanie wykresów

Prezentacja analityczna opiera się na bibliotece `Matplotlib` zintegrowanej bezpośrednio z pętlą zdarzeń PyQt6 za pośrednictwem modułu `FigureCanvasQTAgg`. Pozwoliło to na dynamiczne rysowanie krzywej uczenia się oraz zmęczenia badanego po zakończeniu testów.

3 Dyskusja osiągniętych wyników (Wady i Zalety)

3.1 Zalety napisanej aplikacji

- **Modułowość i rozszerzalność:** Zastosowanie klasy bazowej `BaseTestScreen` ułatwia dodawanie nowych rodzajów testów psychomotorycznych w przyszłości przy minimalnej powtarzalności kodu.
- **Zabezpieczenie przed "falstartem":** Logika flag logicznych skutecznie ignoruje przedwczesne wciśnięcia klawisza, wymuszając rzetelność wyniku.
- **Wysoka ergonomia UI:** Zastosowanie globalnego arkusza stylów (QSS) zapewnia wysoki kontrast i czytelność elementów interfejsu, co jest kluczowe w warunkach testowych (np. brak zmęczenia wzroku).
- **Optymalizacja multimediów:** Wykorzystanie `QSoundEffect` preładuje plik audio do pamięci RAM jako bezstratny strumień PCM, minimalizując opóźnienia sprzętowe.

3.2 Wady i ograniczenia rozwiązania

Należy pamiętać, że aplikacja uruchamiana jest na standardowych komputerach osobistych (PC), a język Python działa pod kontrolą systemu operacyjnego ogólnego przeznaczenia (Windows/Linux/macOS). Oznacza to brak tzw. Czasu Rzeczywistego (RTOS):

- **Opóźnienia narzucane przez OS:** Procesor może w ułamku sekundy przerwać wątek aplikacji, aby zająć się procesem w tle, co wprowadza niekontrolowane zjawisko *jitteru* (opóźnień rzędu kilku milisekund).
- **Opóźnienia sprzętowe (Input/Output Lag):** Czas mierzony przez funkcję programu to czas CPU. Nie uwzględnia on opóźnienia matrycy monitora, czasu odświeżania od momentu wywołania `update()` przez GPU, a także opóźnienia samej klawiatury USB (*polling rate*).

W profesjonalnych, atestowanych urządzeniach medycznych stosuje się mikrokontrolery sprzętowe reagujące na przerwania oraz dedykowane przyciski badawcze. Niemniej jednak, zaimplementowana w języku Python aplikacja stanowi doskonałe, wysoce responsywne przybliżenie sprzętu atestowanego i znakomicie sprawdza się w celach dydaktycznych oraz selekcyjnych o mniejszym rygorze laboratoryjnym.